LUIS ANTÔNIO MAIA SOMBRA

# ARQUITETANDO O NÚCLEO DE DADOS DE UM ECOSSISTEMA DE ANÁLISE SÉRIES TEMPORAIS FINANCEIRAS

RUSSAS
2026

LUIS ANTÔNIO MAIA SOMBRA

# ARQUITETANDO O NÚCLEO DE DADOS DE UM ECOSSISTEMA DE ANÁLISE SÉRIES TEMPORAIS FINANCEIRAS

Trabalho de Conclusão de Curso apresentado ao Curso de Graduação em Engenharia de Software do Campus de Russas da Universidade Federal do Ceará, como requisito parcial à obtenção do grau de bacharel em Engenharia de Software.

UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE RUSSAS
DEPARTAMENTO DE COMPUTAÇÃO
CURSO DE GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

Orientador: Prof. Ms. Pitágoras Graça Martins

RUSSAS
2026

LUIS ANTÔNIO MAIA SOMBRA

# ARQUITETANDO O NÚCLEO DE DADOS DE UM ECOSSISTEMA DE ANÁLISE SÉRIES TEMPORAIS FINANCEIRAS

Trabalho de Conclusão de Curso apresentado ao Curso de Graduação em Engenharia de Software do Campus de Russas da Universidade Federal do Ceará, como requisito parcial à obtenção do grau de bacharel em Engenharia de Software.

Aprovada em: _____

BANCA EXAMINADORA

---

**Prof. Ms. Pitágoras Graça Martins**
(Orientador)
Universidade Federal do Ceará (UFC)

---

**Prof. Dr. Marcos Vinicius de Andrade Lima**
Universidade Federal do Ceará (UFC)

---

**Drª. Rosineide Fernando da Paz**
Universidade Federal do Ceará (UFC)

**Dedicatoria ainda não elaborada**

# Agradecimentos

À minha mãe, Guacira Maria Felipe Maia, que foi meu porto seguro, agradeço por todo o amor e suporte incondicional, e por ter financiado e apoiado minha segunda tentativa em um curso de nível superior.

Ao meu irmão, João Batista, por constituir um exemplo de disciplina a ser seguido, e pelas vezes que me redirecionou ao longo dessa caminhada.

Ao meu padrasto, Beto Cordeiro, por inspirar o espírito de liderança, firmeza e coragem.

Ao Marcos Eugênio de Lima, por me ajudar a atravessar o caminho até a Universidade.

À médica Dra. Célia, pelos anos de assistência e apoio.

À Michelle Guerra Vale, da Assistência Estudantil, por inspirar em mim o espírito de solidariedade, sensibilidade e ética, e pelo apoio prestado ao longo de minha graduação.

Ao Prof. Dr. Marcos Vinicius, por acreditar em mim.

Aos meus amigos do programa bacharelado em Engenharia de Software, da turma de 2022.1 da UFC do Campus de Russas, por toda a alegria que me trouxeram.

Ao Caio Finotti, por me introduzir ao futebol, e ao Tarcísio, por me treinar nesse esporte durante tanto tempo, colaborando para o meu crescimento e saúde.

À Universidade Federal do Ceará, pela segunda chance em um curso de bacharelado, pela transformação cultivada em mim por essa instituição e pelas oportunidades proporcionadas pelo ensino gratuito e de qualidade.

A todos que participaram direta ou indiretamente desta caminhada, caminhada que agora chega ao fim, integrando-se permanentemente à minha história, e abrindo caminho para uma nova fase.

Ao Criador, Deus, que me criou em sua infinita graça, bondade e amor, proporcionando esta experiência, uma nova vida e a renovação da minha mente.

*"Sim, para o jogo da criação, meus irmãos, é preciso um sagrado dizer-sim: o espírito quer agora a sua vontade, o perdido para o mundo conquista o seu mundo"*

*(Friedrich Nietzsche)*

# Resumo

O crescimento das pesquisas em finanças quantitativas no ambiente acadêmico tem evidenciado a necessidade de infraestruturas de dados financeiras robustas e de alto desempenho. No âmbito do projeto Análise de Séries Financeiras, do Laboratório de Tecnologias Inovadoras (LTI) da Universidade Federal do Ceará (UFC), Campus de Russas, a falta de uma solução centralizada obrigava pesquisadores a implementar individualmente conexões incovenientes e relativamente complexas com provedores de mercado. Para resolver esse problema, este trabalho apresenta o projeto e o desenvolvimento da plataforma qDATA, uma infraestrutura de dados financeiros de alta velocidade voltada para a aquisição, o armazenamento e a distribuição de dados históricos e em tempo real (barras OHLC). A ingestão de dados ocorre por meio de uma API REST baseada em FastAPI, que armazena os registros em um banco de dados PostgreSQL com a extensão TimescaleDB, otimizada para séries temporais e empregando estratégia de *upsert*. Para o fornecimento de dados aos consumidores finais com baixa latência, o sistema oferece canais gRPC para contextos de alto desempenho, canais HTTPS para protótipos, além de garantir a segurança e controle de acesso granular via JWT e Supabase. O qDATA compõe um ecossistema de serviços financeiros mais amplo, o LTI FinHub. Esse ecossistema baseia-se em uma arquitetura de microsserviços em contêineres Docker e um cliente front-end web. Em vista disso, o qDATA foi pensado a partir de uma arquitetura que separa responsabilidades, portanto tem enfoque total em ingestão, armazenamento e distribuição. O sistema foi validado através de um dashboard analítico por usuários reais, e em se tratando de eliminação de redundâncias o sistema já abriu caminho para que novos pesquisadores do LTI Finanças realizem seus trabalhos sem a dependência do MetaTrader 5. Alguns exemplos incluem, além do (1) dashboard, o serviço de (2) cálculo e simulação de correlações e o serviço de (3) detecção de anomalias em séries temporais.

**Palavras-chave**: Séries Temporais Financeiras. Arquitetura de Microsserviços. Docker. TimescaleDB. gRPC. gRPC-Web. HTTPS.

# Abstract

The growth of quantitative finance research in academia has highlighted the need for robust, high-performance financial data infrastructures. Within the Financial Time Series Analysis project at the Innovative Technologies Laboratory (LTI) of the Federal University of Ceará (UFC), Russas Campus, the lack of a centralized solution forced researchers to individually implement complex connections with market data providers. To address this issue, this work presents the design and development of the qDATA platform, a high-speed financial data infrastructure aimed at acquiring, storing, and distributing historical and real-time data (OHLC bars). The solution is based on a microservices architecture using Docker containers, comprising extraction modules operating MetaTrader 5 instances via Wine on Linux, integrated with brokerages. Data ingestion is handled through a FastAPI-based REST API, which stores records in a PostgreSQL database equipped with the TimescaleDB extension, optimized for time series data and employing an *upsert* strategy. To deliver data to end consumers with low latency, the system utilizes high-performance gRPC channels, while also ensuring security and granular access control via JWT and Supabase. The architecture adheres to the separation of concerns principle, providing data as a service and isolating the infrastructure from analytical tools. The system was validated through a real analytical dashboard by real users, demonstrating efficiency by eliminating redundancies and allowing researchers to focus on the development of their mathematical models and quantitative strategies.

**Keywords**: Financial Time Series. Microservices Architecture. Docker. TimescaleDB. gRPC.

# Lista de Figuras

Figura 1 – Visão geral da arquitetura: Ingestão distribuída e distribuição segura de dados financeiros. . . . . 22
Figura 2 – Serviços e seus respectivos bancos de dados independentes. . . . . 23
Figura 3 – Visão geral da autonomia dos pesquisadores. . . . . 24
Figura 4 – Banco de usuários no Supabase. . . . . 25
Figura 5 – Demandas do núcleo de dados. . . . . 26
Figura 6 – Ecossistema LTI FinHub . . . . . 30

# Lista de Tabelas

Tabela 1 – Cronograma de execução das atividades do TCC . . . . . 47
Tabela 2 – Cenário 1 — Baixo Volume: latência média agregada (poucos dias). . . 49
Tabela 3 – Cenário 2 — Volume Intermediário: métricas agregadas (301 barras/execução). . . . . 50
Tabela 4 – Cenário 3a — Alto Volume (2 meses): métricas agregadas (16.865 barras/execução). . . . . 50

# Sumário


**1 INTRODUÇÃO . . . . . 21**
**1.1 Escopo . . . . . 25**
**1.2 Objetivos . . . . . 27**
1.2.1 Objetivo Geral . . . . . 27
1.2.2 Objetivos Específicos . . . . . 27
**1.3 Organização do Trabalho . . . . . 28**

**2 FUNDAMENTAÇÃO TEÓRICA . . . . . 29**
**2.1 O Ecossistema LTI FinHub . . . . . 29**
**2.2 Arquitetura de Microsserviços . . . . . 30**
**2.3 Coleta de Dados com Tickers Especializados . . . . . 31**
**2.4 Contêineres e Docker . . . . . 32**
2.4.1 Compatibilidade de Binários em Ambientes Linux (Wine) . . . . . 32
**2.5 Separação de Preocupações (Separation of Concerns) . . . . . 32**
**2.6 Protocolos de Comunicação: REST/HTTP e gRPC . . . . . 33**
**2.7 Bancos de Dados para Séries Temporais: TimescaleDB . . . . . 34**
**2.8 FastAPI e Programação Assíncrona (ASGI) . . . . . 34**
**2.9 Estratégia de Upsert e Idempotência . . . . . 35**
**2.10 Autenticação e Autorização com JWT e Supabase . . . . . 36**
**2.11 Desenvolvimento Incremental e Integração Contínua . . . . . 36**

**3 METODOLOGIA . . . . . 39**
**3.1 Visão geral da metodologia . . . . . 39**
**3.2 Questões de pesquisa . . . . . 39**
**3.3 Fontes de conhecimento e delimitação do estudo . . . . . 40**
**3.4 Desenvolvimento incremental da solução . . . . . 41**
3.4.1 Especificação do Contrato de Dados . . . . . 41
**3.5 Governança . . . . . 43**
3.5.1 Versionamento de API . . . . . 43
3.5.2 Políticas de Acesso por Perfil . . . . . 43
3.5.3 Ownership dos Microsserviços . . . . . 43
3.5.4 Gestão de Mudança . . . . . 43
**3.6 Técnicas e testes aplicados . . . . . 44**
**3.7 Métricas de avaliação . . . . . 44**
**3.8 Procedimento experimental . . . . . 44**
**3.9 Ameaças à validade . . . . . 45**

**3.10 Síntese** . . . . . **45**

**4 CRONOGRAMA** . . . . . **47**

**5 RESULTADOS** . . . . . **49**
**5.1 Análise quantitativa de *performance* de comunicação** . . . . . **49**
5.1.1 Cenário 1 — Baixo Volume . . . . . 49
5.1.2 Cenário 2 — Volume Intermediário (Aleatório A) . . . . . 49
5.1.3 Cenário 3a — Alto Volume (Janela de 2 meses) . . . . . 50
**5.2 Avaliação Qualitativa dos Protocolos de Comunicação** . . . . . **51**
5.2.1 Adoção pelos Usuários e Impacto Prático . . . . . 51
5.2.2 Impactos Organizacionais e na Infraestrutura . . . . . 51
5.2.3 Experiência adquirida . . . . . 52

**6 CONCLUSÕES E TRABALHOS FUTUROS** . . . . . **53**

# 1 INTRODUÇÃO

O crescimento das pesquisas em finanças quantitativas nas universidades brasileiras tem evidenciado a necessidade de infraestruturas de dados robustas e de alto desempenho. No âmbito do projeto **Análise de Séries Financeiras**, vinculado ao Laboratório de Tecnologias Inovadoras (LTI) da Universidade Federal do Ceará (UFC), *campus* de Russas, pesquisadores e estudantes necessitam de acesso contínuo e confiável a séries temporais de ativos financeiros — cotações históricas e em tempo real de ações, câmbio e outros instrumentos — para viabilizar pesquisas de modelos quantitativos. Além disso, a obtenção de dados de maneira *headless* (sem interface gráfica interativa) torna-se uma exigência em projetos com uma quantidade significativa de membros.

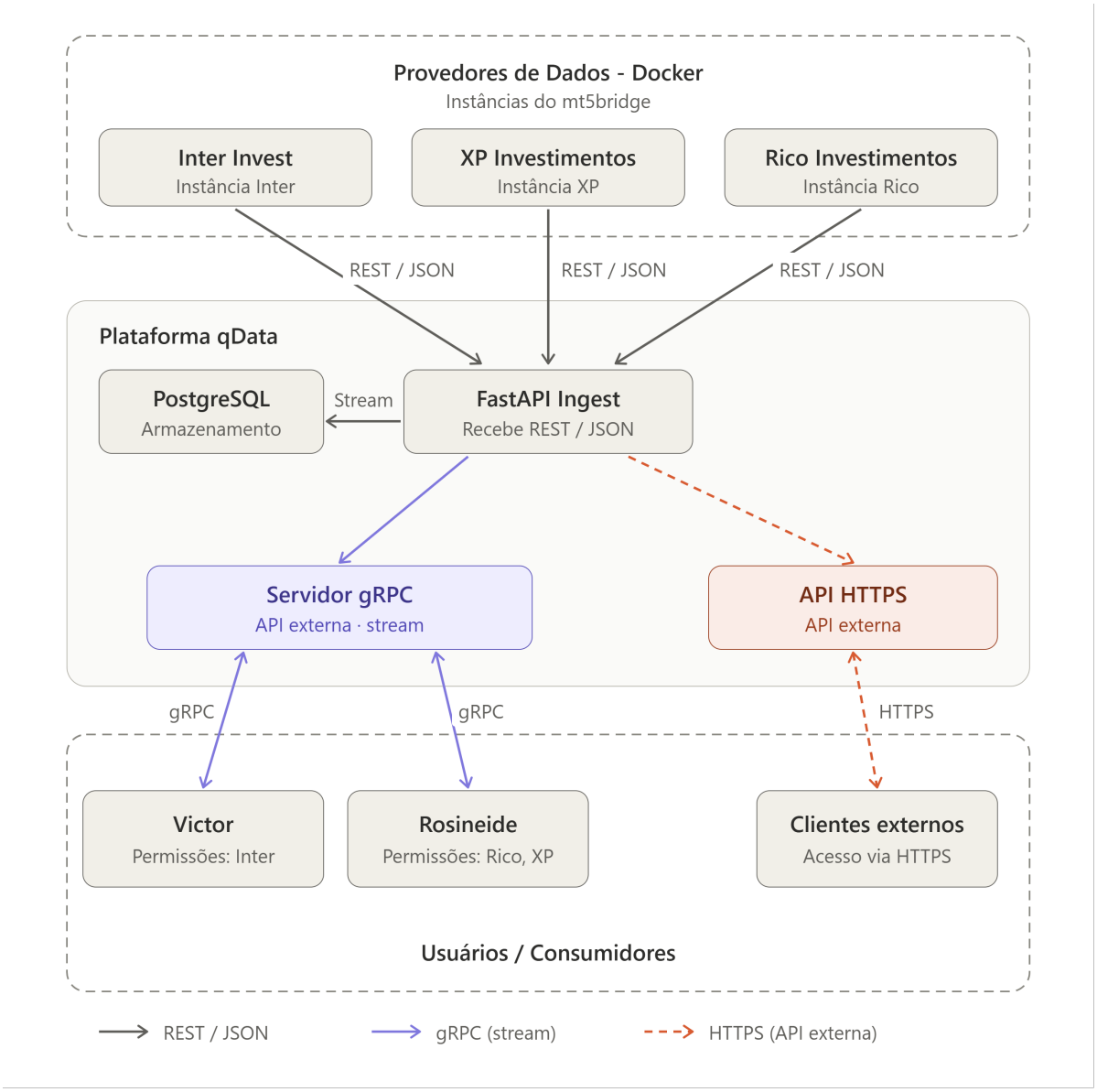
A ausência de uma infraestrutura centralizada impõe custos operacionais substanciais. Individualmente, cada pesquisador precisaria implementar conexões com provedores de dados externos (*web-sockets*, dependências de bibliotecas, etc.), gerenciar credenciais de corretoras, lidar com falhas de conectividade e manter códigos de coleta que fogem do escopo de sua especialidade científica. Em face dessa problemática, estabelecemos a seguinte questão norteadora para o presente trabalho: *Como fornecer serviços de dados de séries temporais para o LTI de forma centralizada, garantindo alto desempenho, consistência dos dados, alta disponibilidade do serviço e controle de acesso seguro, sem que cada pesquisador precise implementar suas próprias integrações?* Traçamos essa pergunta-problema visando como resposta uma solução-produto de potencial econômico real, isto é, com valor percebido por pessoas interessadas (*stakeholders*) na área quantitativa do mercado financeiro.

Para solucionar essa questão, surgiu a necessidade de desenvolver uma plataforma que atendesse às demandas de discentes e docentes quanto ao acesso a dados históricos e em tempo real. Essas demandas estruturam-se em requisitos funcionais (obtenção de cotações, volume transacionado, ordens agendadas) e não funcionais (alto desempenho, consistência dos dados mantida sob alta taxa de ingestão, mecanismos iniciais de disponibilidade do serviço e controle de acesso granular). A partir dessa perspectiva, iniciamos a concepção de um núcleo *headless* e centralizado de dados financeiros.

Diante disso, desenhamos um "ponto de partida"que consistia, inicialmente, em uma simples API de séries temporais, mas que evoluiu, a nível de produto, para um ecossistema de serviços de dados financeiros. A afirmação desta evolução para um "ecossistema"justifica-se pelo fato de que a solução proposta iniciou-se como (1) um projeto técnico restrito a um núcleo de obtenção e distribuição de dados (Nucleo de dados), expandiu-se para (2) um escopo de serviço de software completo que interage com clientes e outros serviços (coordenação com serviços-chave), coordenação técnico (Ecossistema e interfaces),e por fim os aspectos de (3) gestão do projeto enquanto ecossistema (Gerenciamento do projeto).

Para concretizar essa visão, pensamos um núcleo de armazenamento e distribuição baseada em *containers* Docker. O núcleo de aquisição de dados, implementado de forma independente pelo discente Evamberto Oliveira, executa o *MetaTrader 5* integrado a provedores externos, como as corretoras Rico, XP e Inter. Esse serviço de aquisição, denominado *mt5bridge*, envia os dados dos ativos para um canal de recebimento de dados do qData, que então os persiste no banco de dados. Toda a ingestão ocorre via Representational State Transfer (REST)/HTTPS, justificável pelo fluxo relativamente baixo de entrada de dados, o que evidencia a coordenação necessária entre o mt5BRIDGE e o qDATA.

Figura 1 – Visão geral da arquitetura: Ingestão distribuída e distribuição segura de dados financeiros.

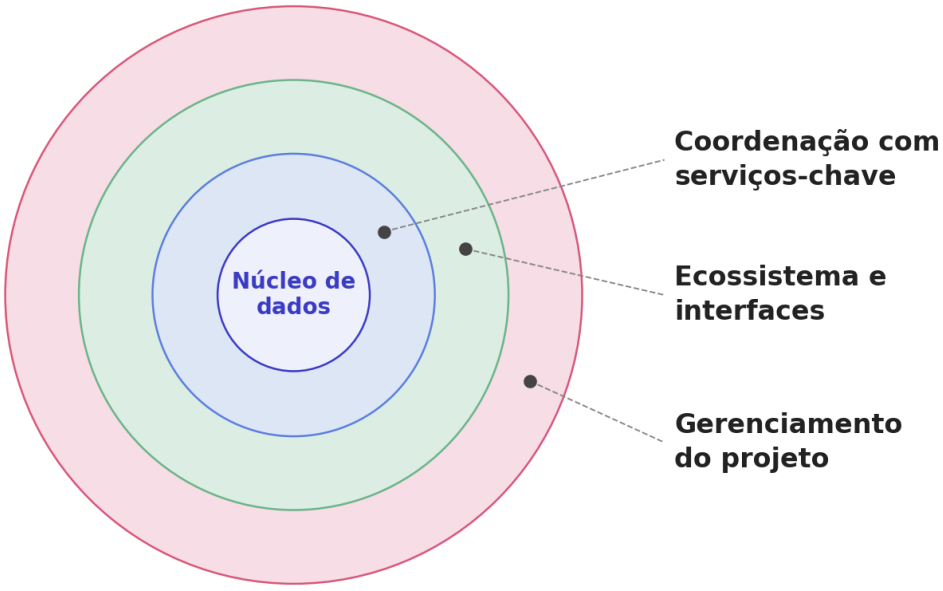



A persistência no qDATA é gerenciada por um banco de dados PostgreSQL com

estratégia de *Upsert* (Update + Insert). Essa abordagem previne duplicidades e mantém a consistência das informações mesmo sob alta taxa de ingestão simultânea. Ademais, o armazenamento é otimizado para séries temporais por meio do uso de hyper-tabelas da extensão TimeScale.

No que tange à distribuição técnica dos dados, o protocolo *gRPC Remote Procedure Call* (gRPC) permite o estabelecimento de conexões contínuas (com sessão) e uma obtenção mais rápida, sendo útil em consultas massivas, como o resgate de dados correspondentes a um ano inteiro. Contudo, visando complementar e facilitar essa distribuição, implementou-se também uma interface HTTPS/2. Essa segunda opção foi projetada para que a conexão com a API pudesse ser realizada de forma mais ágil pelos usuários (pesquisadores e desenvolvedores), uma vez que o uso de HTTPS apresenta uma curva de aprendizado substancialmente menor e um *overhead* de desenvolvimento reduzido.

Figura 2 – Serviços e seus respectivos bancos de dados independentes.

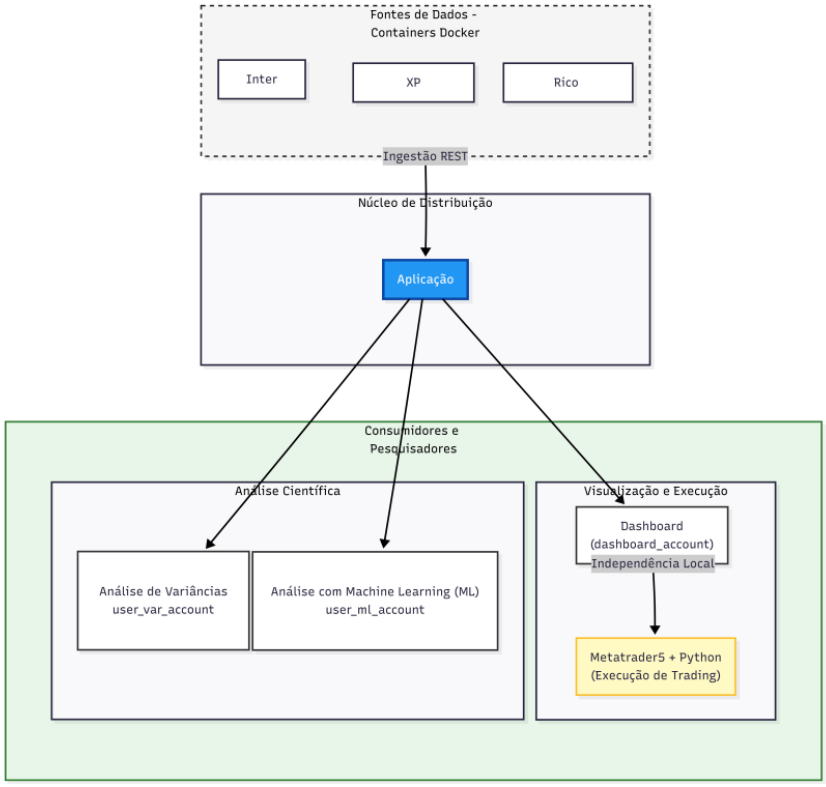



À medida que a ferramenta avançou em produção, membros do projeto passaram a utilizá-la como base para construir serviços quantitativos auxiliares, tais como cálculos de correlações com *bootstrapping*, experimentos em Python de trabalhos teóricos conduzidos no projeto, e rotinas de identificação de anomalias. Esse cenário reforça o desenho de um ecossistema coeso de serviços e análises de apoio.

Por fim, cabe ressaltar que a plataforma qData é uma ferramenta de uso estritamente interno ao Laboratório de Tecnologias Inovadoras (LTI). Ela não configura um serviço comercial de distribuição de dados de mercado para terceiros externos. Assim, estabelecemos uma rigorosa política de não revenda, não sublicenciamento e não disponibilização pública dos dados obtidos. Assim, é necessária a realização de um estudo sobre o uso da fonte de dados (XP Invesimtimentos) pelo LTI (ou LTI Finanças) enquanto entidade jurídica singular. Esse requisito de compliance reflete-se no gerenciamento granular de permissões de acesso, que foi configurada de maneira detalhada em um banco de gerenciamento de acessos feitos no Supabase. A implantação de contêineres individuais para aquisição de dados será, futuramente, discutida com o responsável pelo mt5bridge.

A arquitetura do ecossistema de microsserviços promove o princípio de **Separação de Responsabilidades** (*Separation of Concerns*): o qData concentra-se exclusivamente no fornecimento dos dados como serviço compartilhado, enquanto as ferramentas dos pesquisadores — painéis analíticos, algoritmos de ordens de *trading* e modelos estatísticos — focam em suas lógicas específicas. Isso elimina acoplamentos indesejados e permite que diferentes membros do LTI e projetos da UFC utilizem essa infraestrutura de dados sem redundâncias.

Figura 3 – Visão geral da autonomia dos pesquisadores.



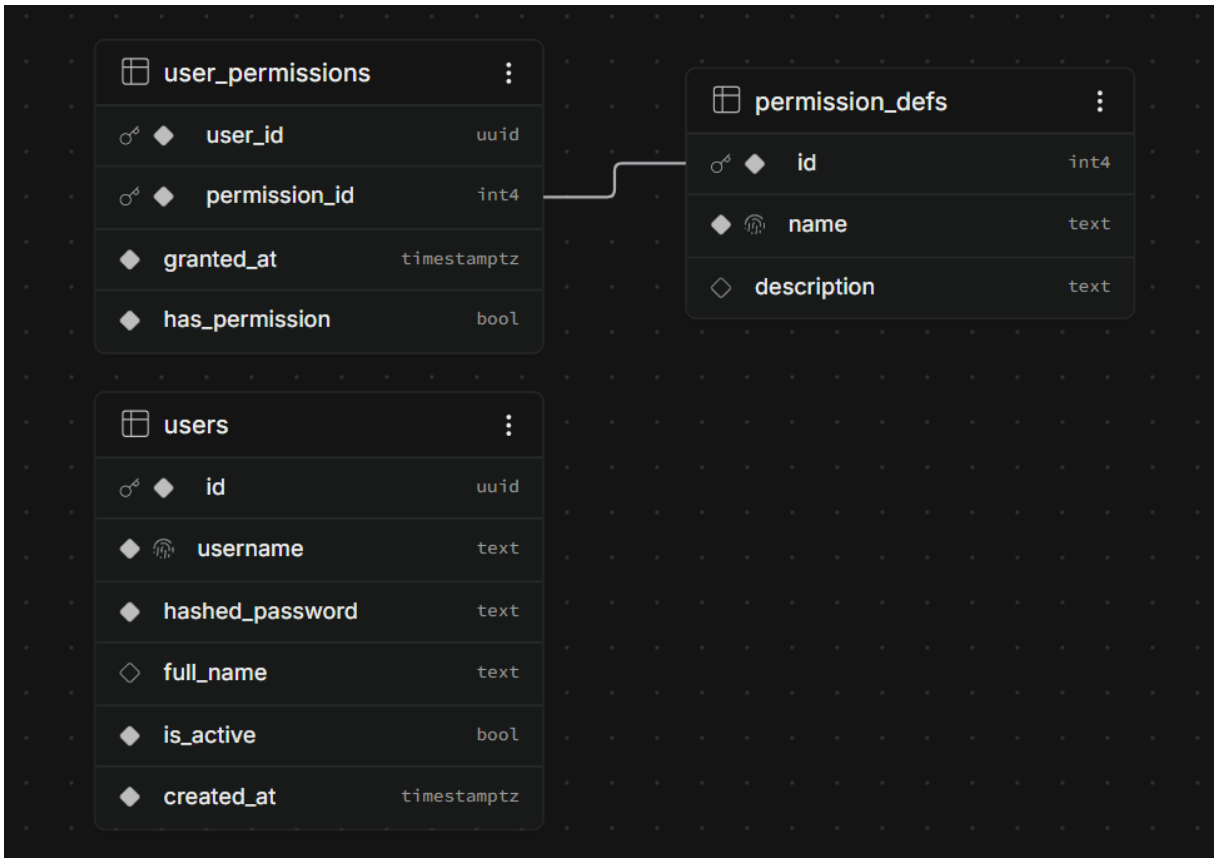

Nesse sentido, a infraestrutura para estudos e experimentos que eventualmente exijam obtenção de dados com latências da ordem de micro-segundos ou milissegundos será objeto de um novo serviço. Este tipo de aplicação precindiria de um núcleo de distribuição, pois

os dados seriam obtidos diretamente do agente de aquisição de dados para evitar latência adicional de um núcleo de distribuição.

Deve-se observar, no entanto, que o uso em outros projetos requer, por questões legais e de governança, a execução de uma nova instância isolada do sistema, haja vista as restrições à distribuição. Nesse contexto de separação de responsabilidades, a execução de ordens de compra e venda, por exemplo, permanece uma tarefa local e autônoma de cada ferramenta consumidora, sem interferência direta com a camada de ingestão remota ou de distribuição de dados. Tanto por questões legais quanto de engenharia de software, decidiu-se consolidar o ecossistema sob o escopo do *LTI FinHub*.

O acesso à plataforma é protegido por autenticação baseada em *JSON Web Token* (JWT) integrada ao Supabase, viabilizando um controle de acesso granular por perfil de usuário e por provedor de dados.

Figura 4 – Banco de usuários no Supabase.



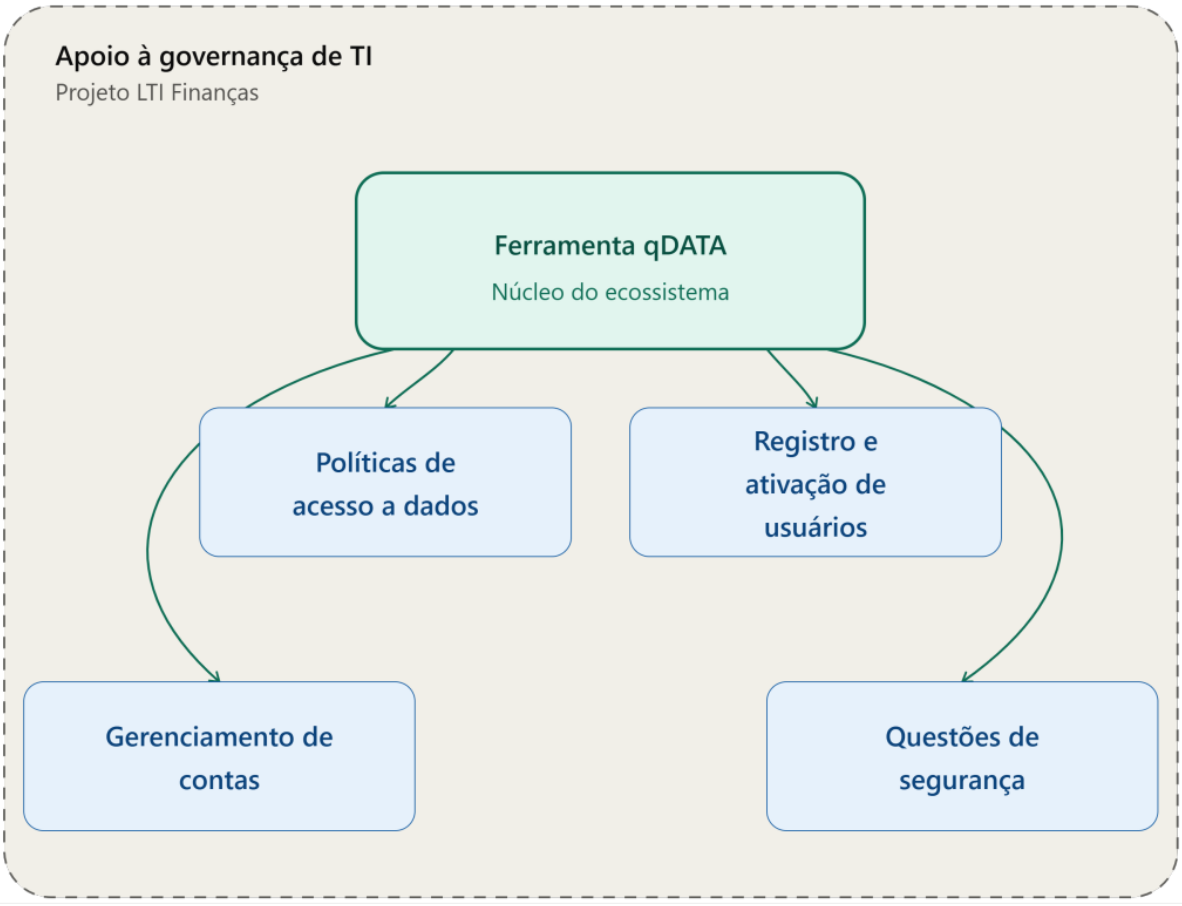

## 1.1 Escopo

A plataforma qDATA abrange a ingestão de dados de corretoras selecionadas, o armazenamento e a distribuição desses dados para consumidores internos do grupo de pesquisa LTI, com foco no projeto *Análise de Séries Financeiras*. Dentro do escopo técnico

estão a comunicação com um software de aquisição de barras *Open, High, Low, Close* (OHLC), o armazenamento otimizado com hyper-tabelas através do TimeScale — o que viabiliza a entrega de dados em alta velocidade aos consumidores autorizados — e a disponibilização desses dados via HTTPS, visando cenários que exigem praticidade e agilidade no desenvolvimento de aplicações.

Ao longo deste trabalho, também são analisadas as políticas de acesso a dados, o registro e ativação de usuários, o gerenciamento de contas do projeto e outras questões de segurança que impactam diretamente a ferramenta qDATA e o ecossistema como um todo. Assim, partindo da ferramenta qDATA como elemento central, o escopo se expande radialmente para uma visão arquitetural mais ampla, englobando o apoio ao gerenciamento do projeto *LTI Finanças*.

Figura 5 – Demandas do núcleo de dados.




Por outro lado, estão categoricamente fora do escopo desta plataforma: a execução de ordens de compra e venda nas corretoras, a gestão de risco financeiro e o desenvolvimento de interfaces de *trading* para usuários finais. O qData atua exclusivamente como infraestrutura de dados, cabendo à lógica de análise e negociação permanecer sob a estrita responsabilidade de cada ferramenta ou pesquisador consumidor.

## 1.2 Objetivos

### 1.2.1 Objetivo Geral

Projetar e implementar uma plataforma de dados financeiros centralizada e orientada à disponibilidade para o projeto *Análise de Séries Financeiras* do LTI.

### 1.2.2 Objetivos Específicos

- Consumir a plataforma com os *containers* Docker de aquisição de dados do MetaTrader 5 (MT5), os quais operam em ambiente Linux utilizando Wine;
- Desenvolver o núcleo de ingestão e armazenamento de dados empregando o *framework* FastAPI e o banco de dados PostgreSQL, mediado por protocolo HTTPS REST;
- Implementar serviços de distribuição de dados aos consumidores utilizando os protocolos gRPC, gRPC-web e HTTPS;
- Estabelecer mecanismos iniciais para a garantia de disponibilidade, abrangendo a implementação de testes unitários (via `pytest`) e a implementação de fluxos de integração contínua (GitHub Actions), que utilizam estes testes.
- Comparar o desempenho das interfaces de distribuição (gRPC, gRPC-Web e HTTPS) em termos de latência e consumo de banda
- Documentar formalmente as políticas de controle de acesso e versionamento como prática basilar de governança de TI;

## 1.3 Organização do Trabalho

Este trabalho está organizado em seis capítulos, estruturados da seguinte forma:

- **Capítulo 1 – Introdução:** Apresenta a contextualização do problema enfrentado pelos pesquisadores do Laboratório de Tecnologias Inovadoras (LTI) no acesso a dados financeiros. Expõe a justificativa para a criação da plataforma, definindo a questão norteadora, o escopo do projeto e os objetivos geral e específicos a serem alcançados.

- **Capítulo 2 – Fundamentação Teórica:** Aborda os conceitos teóricos que embasam as decisões arquiteturais da plataforma. São explorados temas como microsserviços, estratégias de ingestão e armazenamento de grandes volumes de dados e modelagem de séries temporais. Além disso, detalha as tecnologias adotadas, como *containers* Docker, o banco de dados PostgreSQL com a extensão TimeScale, além dos protocolos de comunicação gRPC, gRPC-Web e REST/HTTPS.

- **Capítulo 3 – Metodologia:** Detalha o processo de desenvolvimento do sistema de forma pragmática, desde a concepção arquitetural até a implementação. O capítulo descreve a modelagem do sistema, a implementação da persistência (utilizando hyper-tabelas e estratégia de *Upsert*), a construção das interfaces de distribuição, além da estruturação dos testes automatizados e fluxos de integração contínua (CI).

- **Capítulo 4 – Cronograma:** Apresenta o planejamento temporal de todas as atividades envolvidas no trabalho. Expõe como as etapas de levantamento de requisitos, implementação das funcionalidades, testes, validação e escrita do documento foram distribuídas ao longo dos meses de execução do projeto.

- **Capítulo 5 – Resultados:** Expõe e discute os resultados obtidos com o funcionamento do sistema em produção. O capítulo engloba avaliações e métricas de desempenho, comparando os meios de distribuição (gRPC, gRPC-Web e HTTPS). Também detalha o sistema de segurança e controle granular de acessos (via Supabase) e relata a efetiva adoção da plataforma pelos pesquisadores do ecossistema do LTI Finanças.

- **Capítulo 6 – Conclusão:** Delineia as considerações finais sobre o trabalho realizado, verificando o cumprimento dos objetivos propostos. Discute as limitações técnicas ou organizacionais da solução desenvolvida e enumera propostas de melhorias e trabalhos futuros para a evolução contínua da infraestrutura de dados.

# 2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo tem como objetivo apresentar os conceitos teóricos, padrões arquiteturais e tecnologias que embasam o desenvolvimento da plataforma qDATA. Ao longo das próximas seções, discutiremos as bases fundamentais necessárias para a compreensão das escolhas metodológicas detalhadas posteriormente, contextualizando-as no cenário de engenharia de software voltada a sistemas de alta disponibilidade e baixa latência.

Dada a natureza do problema do LTI, que exige a ingestão contínua de grandes volumes de dados financeiros (barras OHLC) e seu consumo em atividades internas do LTI, a infraestrutura projetada baseia-se fortemente em práticas modernas da indústria. Discorreremos sobre como conceitos consolidados, como microsserviços, conteinerização e comunicação híbrida via protocolos eficientes, sustentam os requisitos operacionais sem comprometer a estabilidade do serviço.

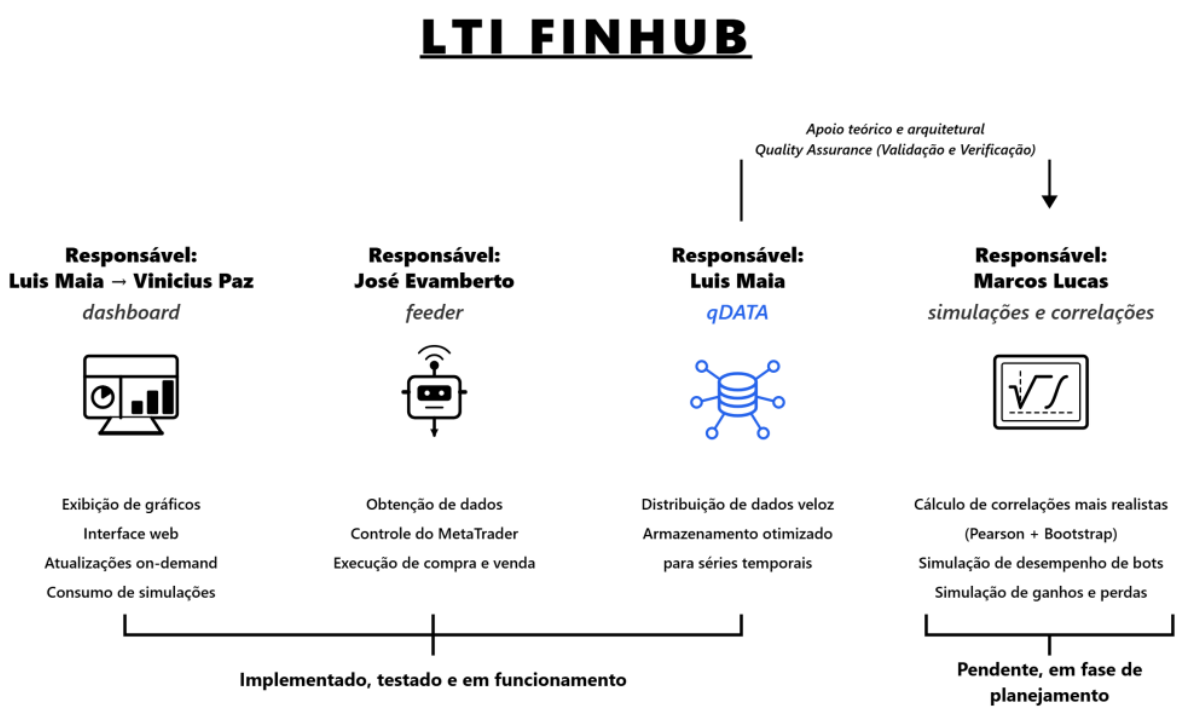
## 2.1 O Ecossistema LTI FinHub

O FinHub é um ecossistema abrangente desenvolvido no âmbito do Laboratório de Tecnologias Inovadoras (LTI). Trata-se de um sistema amplo que orquestra múltiplos serviços e componentes para viabilizar a pesquisa quantitativa avançada. A Figura 6 ilustra a arquitetura conceitual e a interação entre os diversos elementos que compõem este ambiente.

Neste trabalho, adotamos como ponto de partida inicial o componente qDATA, dedicando a ele um enfoque predominantemente técnico. Contudo, conservamos a perspectiva sistêmica de que o qDATA, embora seja um componente individual, desempenha um papel central e, portanto, precisa se relacionar adequadamente com os demais serviços do ecossistema. O sucesso de sua implementação depende da interoperabilidade harmoniosa com módulos como o *feeder*, os painéis de visualização (*dashboards*) e os futuros serviços de simulação.

Ademais, ressalta-se que o presente estudo consolida as bases infraestruturais. Um trabalho futuro (TCC2) dará ênfase a aspectos mais amplos, como governança de TI, aprofundamento na arquitetura de microsserviços e práticas de *Product Ownership*, garantindo que a evolução do ecossistema FinHub ocorra de forma gerenciada, escalável e alinhada às necessidades estratégicas dos pesquisadores envolvidos.

Figura 6 – Ecossistema LTI FinHub




## 2.2 Arquitetura de Microsserviços

A arquitetura de microsserviços consiste em uma abordagem na qual uma única aplicação é composta por um conjunto de pequenos serviços, cada um executando seu próprio processo e comunicando-se por meio de mecanismos leves, geralmente *Application Programming Interfaces* (APIs) sobre o protocolo *Hypertext Transfer Protocol* (HTTP). Conforme postulado por **??**), essa arquitetura enfatiza serviços independentemente implantáveis, construídos em torno de capacidades de negócios e com implantação automatizada.

Ao contrário de abordagens monolíticas – onde todos os componentes lógicos (interface, regras de negócio e acesso a dados) são desenvolvidos dentro de um mesmo repositório – os microsserviços promovem um acoplamento mais fraco, e melhora a coesão. Evidentemente, essa melhoria não é absoluta, pois o desmembramento de um software grande em unidades menores pode impactar contratos, dependências, segurança ou a integração com outros serviços. Ainda assim, as vantagens citadas ultrapassa a mera organização arquitetural do código.

Nesse sentido, conforme visto nos trabalhos dentro do laboratório, tal abordagem permitiu mais flexibilidade para a equipe, e membros da equipe podem se voluntariar a serem responsáveis por um componente específico. Uma prática interessantíssima nesse cenário é a atribuição a um membro de responsabilidade "end-to-end"por algum(s) serviço(s).

Em outras palavras, é possível estruturar a organização do laboratório de modo que certos colaboradores sejam totalmente responsáveis por um serviço. Tal trabalho é, ainda, viabilizado pelo tamanho do serviço, que devido à modularização não representa um software inteiro, mas apenas um parte do todo.

No contexto deste trabalho, a arquitetura de microsserviços justifica a divisão estrutural entre o componente de coleta de dados (*feeder*) e a plataforma centralizadora (qDATA). Ou seja, foi projetado para que as responsabilidades de (1) aquisição das séries temporais e de (2) armazenamento e distribuição sejam escaladas, monitoradas e mantidas de maneira totalmente autônoma.

Nesse contexto, destaca-se que uma versão inicial – e precária – do *feeder* foi prototipada e usada em experimentos no projeto, mas a versão final e de qualidade só foi desenvolvida posteriormente, pelo discente José Evamberto Lima Oliveira, conforme descrito em seu trabalho *Manutenção e Evolução do Sistema MT5-Websocket-Bridge no projeto de extensão LTI – Análise de séries financeiras* (**??**).

## 2.3 Coleta de Dados com Tickers Especializados

A especialização em coleta de dados, por parte do discente José Oliveira, cujo trabalho se comunica diretamente com o meu, abriu caminho para três possibilidades de trabalho. Em primeiro lugar, possibilitou a divisão da carga de trabalho, permitindo que o trabalho do sistema de dados fosse dividido em duas partes menores, evitando a sobrecarga laboral.

Em segundo lugar, tornou possível a coleta especializada de tickers. Nesse sentido, cita-se o fato de que há dois tipos de tickers na biblioteca Python do Metatrader5. São eles: (1) `mt5.COPY_TICKS_TRADE`, que filtra a base de dados do MT5 e só te entrega os momentos em que um comprador e um vendedor fecharam negócio.

Em segundo lugar, temos o (2) `mt5.COPY_TICKS_INFO`, que traz apenas as atualizações do Book de Ofertas (Livro de Preços). Ou seja, alguém colocou, moveu ou cancelou uma ordem limite de compra (Bid) ou venda (Ask), mas nenhum negócio foi fechado. Ele é fortemente recomendado para trabalhos estatísticos, simulações, uso em estratégias de *trading*, etc.

Por fim, temos o tipo de ticker (3) `mt5.COPY_TICKS_ALL` que traz absolutamente tudo. Cada vez que o Bid mudou, o Ask mudou, ou um negócio aconteceu, uma nova linha é gerada. Deve-se notar que o volume de dados gerado por essa flag é gigantesco, especialmente em ativos líquidos como mini-índice (WIN), um tipo ideal para trabalhos e técnicas estatísticas que dependem muito de volume.

Nesse contexto, no que tange aos *stakeholders* do projeto, temos a possibilidade de ofertar à equipe e à coordenadora do projeto três tipos diferentes de dados para um mesmo ativo, que devem ser escolhidos conforme a necessidade de cada demanda. Concluindo, o

trabalho realizado em cada um desses componentes se torna mais especializado, produzindo componentes de mais qualidade no produto final.

## 2.4 Contêineres e Docker

A conteinerização revolucionou a implantação de software ao oferecer uma virtualização em nível de sistema operacional (processo). O Docker despontou como a principal ferramenta para essa prática, permitindo que desenvolvedores empacotem uma aplicação com todas as suas dependências, bibliotecas e arquivos de configuração necessários em um único artefato portável (**??**).

Um dos maiores ganhos operacionais da utilização de contêineres é o princípio conhecido como "*build once, run anywhere*"(construa uma vez, execute em qualquer lugar). Na prática, se o contêiner funciona no ambiente de desenvolvimento, seu comportamento será idêntico nos ambientes de teste, homologação e produção, erradicando a clássica síndrome do "na minha máquina funciona". Essa previsibilidade minimiza substancialmente o esforço logístico em implantações complexas.

No desenvolvimento do projeto, o Docker permite empacotar o código do banco de dados e o código do ASGI (FastAPI) sem expor a máquina hospedeira a configurações conflitantes, além de permitir rápida manutenção do banco de dados e *deploys* mais seguros.

### 2.4.1 Compatibilidade de Binários em Ambientes Linux (Wine)

Ainda dentro do domínio da infraestrutura de contêineres, surge frequentemente a necessidade de compatibilizar binários compilados exclusivamente para a plataforma Windows em ambientes operacionais baseados em Linux. O MT5, terminal frequentemente mandatório para conexão com corretoras brasileiras (como XP, Rico e Clear), possui versões nativas unicamente para o ecossistema da Microsoft.

Para contornar tal barreira no Linux, convenciona-se utilizar a camada de compatibilidade Wine (*Wine Is Not an Emulator*). Trata-se de um software que traduz as chamadas de sistema (API) do Windows para equivalentes no padrão *Portable Operating System Interface* (POSIX) do Linux em tempo real, evitando os atrasos de desempenho característicos de emuladores convencionais ou máquinas virtuais pesadas (**??**).

## 2.5 Separação de Preocupações (Separation of Concerns)

O princípio da Separação de Preocupações, originário dos primórdios da Ciência da Computação, determina que um sistema deve ser estruturado em seções distintas, onde cada seção lida com um conjunto específico e delineado de informações e tarefas. Um módulo deve ter uma, e apenas uma, razão primária para sofrer modificações.

Em sistemas distribuídos modernos, aplicar esse princípio significa evitar componentes "Deus", isto é, objetos ou serviços que sabem e fazem demais, um fenômeno discutido nas aulas de Programação Orientada a Objetos. Ao delegar as preocupações de obtenção de dados de mercado para um sistema dedicado, libera-se a ferramenta consumidora da tarefa de gerenciar autenticações complexas de API ou manter a resiliência de *sockets* contínuos, focando unicamente na análise dos dados em si. Mais do que isso: essa arquitetura permitiu delegar a responsabilidade do "Departamento de Aquisição de Dados"(figura de linguagem) a um gerente dedicado especificamente a isso.

No domínio do Laboratório de Tecnologias Inovadoras, a adoção incisiva deste conceito molda a justificativa central do trabalho. O qDATA concentra-se puramente em agir como infraestrutura de dados compartilhada (*Market Data*). A lógica de análise quantitativa, o treinamento de modelos estocásticos e a execução de ordens nas corretoras permanecem estritamente sob responsabilidade do cliente (*researcher*). Essa fronteira rígida protege o núcleo do sistema de *bugs* advindos de modelos preditivos em fase de experimentação.

## 2.6 Protocolos de Comunicação: REST/HTTP e gRPC

A intercomunicação eficaz é o coração de qualquer arquitetura de serviços distribuídos. O REST (*Representational State Transfer*), introduzido por **??**), tornou-se o padrão *de facto* na web, caracterizado por sua operação *stateless* sobre HTTP e pela ampla adoção global por sua clareza de uso. Contudo, em virtude de tradicionalmente carregar respostas verbosas em texto *JavaScript Object Notation* (JSON) e depender de conexões repetidas no HTTP/1.1, a comunicação REST costuma esbarrar em gargalos quando exigida baixa latência e alta vazão (milhões de eventos por minuto).

Em contrapartida, o gRPC (*gRPC Remote Procedure Call*) é um *framework* moderno, focado em alta performance, que utiliza o protocolo HTTP/2 subjacente e o formato de serialização binária Protocol Buffers (Protobuf). Ele viabiliza não apenas chamadas incisivamente menores e mais rápidas no envio por rede, como também introduz o conceito valioso de conexões multiplexadas e o *streaming* bidirecional de mensagens contínuas, mantendo o canal perenemente aberto (**??**). Em outras palavras, conseguimos enviar múltiplas requisições por um mesmo canal *Transmission Control Protocol* (TCP) em paralelo de um ponto de vista lógico, aumentando a performance da rede, e delegando a responsabilidade de controle de fluxo para o protocolo HTTP/2, que é otimizado para esse tipo de comunicação.

Adotamos, portanto, uma arquitetura de rede híbrida alinhada com as melhores práticas da indústria financeira. A ingestão de barras (*feeder* enviando ao núcleo) ocorre via *endpoints* REST baseados no HTTP clássico, permitindo fácil inspeção, monitoramento com *webhooks* e integração simplificada. Simultaneamente, a distribuição dos dados temporais para o painel (*dashboard*) dos pesquisadores é trafegada via canais gRPC otimizados,

superando obstáculos inerentes ao JSON e minimizando fortemente a latência nas consultas volumosas de cotações históricas. Esclarecemos que a melhoria feita ao usar Protocol Buffers (Protobuf) em vez de JSON consiste no fato que esse tipo de serialização é mais compacta, além de ser mais rápida para serializar e desserializar em comparação ao formato de texto (JSON, *Extensible Markup Language* (XML), etc). Nesse sentido, definimos serialização e desserialização como o processo de converter um objeto entre um formato utilizável pela memória do computador e um formato utilizável para transmissão ou armazenamento.

## 2.7 Bancos de Dados para Séries Temporais: TimescaleDB

Dados financeiros em formato OHLC (*Open, High, Low, Close*) apresentam o caráter uníssono e linear de séries temporais (*time-series*). Os bancos de dados relacionais tradicionais, embora robustos nas consultas transacionais simples e cruzamentos complexos (JOINs), frequentemente demonstram acentuada degradação em desempenho e crescimento descontrolado no espaço de armazenamento físico à medida em que as tabelas acumulam centenas de milhões de pontos sequenciais de observação no tempo.

A extensão de Postgres denominada "TimescaleDB"se projeta essencialmente como uma ferramenta voltada à adequação do Sistema de Gerenciamento de Banco de Dados (SGBD) padrão PostgreSQL aos exigentes critérios de *Time-Series Data*. Seu principal diferencial repousa nas chamadas *hypertables*: na superfície atuam identicamente como tabelas relacionais clássicas, no entanto operam no interior criando e gerenciando o particionamento dinâmico automático em blocos temporais isolados (**??**).

Desta forma, os pesquisadores do LTI podem submeter requisições envolvendo densas estatísticas com cálculos de médias e consolidação dos volumes intradiários em escalas cronológicas maiores (através das funções nativas como `time_bucket`). O mecanismo subjacente somente inspeciona as frações de dados contidas nos intervalos requeridos (sem necessidade de percorrer todos os registros contidos em base de dados), e utiliza ainda algoritmos de compressão columnar sofisticados que propiciam economia substancial do espaço em disco sem sacrifício da presteza das consultas.

## 2.8 FastAPI e Programação Assíncrona (ASGI)

Com o advento e adoção crescente de ambientes concorrentes de *Input/Output* (I/O), o padrão Python *Asynchronous Server Gateway Interface* (ASGI) (*Asynchronous Server Gateway Interface*) ascendeu para solucionar o bloqueio síncrono que assolava seus antecessores (como o Web Server Gateway Interface). O padrão ASGI gerencia requisições paralelamente de forma assíncrona, o que significa que a *thread* do servidor pode atender a outras chamadas web sem travar a execução enquanto aguarda por respostas de tráfego de redes externas ou resoluções de consultas morosas no banco de dados.

O *framework* FastAPI constrói-se exatamente sob essa base arquitetural assíncrona, ancorando suas credenciais de escalabilidade. Notabilizado pelo elevado e provado desempenho — equiparável a *frameworks* baseados em Node.js e até mesmo linguagens compiladas como Go —, o FastAPI inclui documentação automatizada nativa (Swagger UI e ReDoc), e integra de maneira transparente a validação estrita de parâmetros das rotas por meio do robusto ecossistema de validação e tipagem "Pydantic"(**??**).

Na engenharia do qDATA, essa capacidade de escalar perante tráfego denso foi determinante na seleção do FastAPI para hospedar a camada REST *controller* do microsserviço central. O fluxo ininterrupto de dezenas (ou centenas) de *feeders* simultâneos despejando as atualizações do índice a cada minuto causaria um enfileiramento nocivo numa estrutura web linear, ao passo que em regime assíncrono o banco de dados e a rede resolvem as requisições recebidas com sobreposição altamente otimizada. Cabe destacar que embora o serviço de distribuição de dados tenha sido feito com gRPC, a alimentação dos dados foi implementada via *endpoints* REST/HTTP de recepção de dados. Essa escolha foi feita por questões de tempestividade do projeto, além do fato de que o gargalo desse ecossistema é a distribuição, e não a ingestão.

## 2.9 Estratégia de Upsert e Idempotência

Sistemas distribuídos convivem invariavelmente com o modelo de entrega na rede conhecido como "pelo menos uma vez"(*at-least-once*), o que indica que repetições não intencionais de requisições ou atrasos são corriqueiros mediante instabilidades de conectividade. Para ingerir ininterruptamente um extenso número de barras de ativos e neutralizar os duplos envios na rede ou reconexões espontâneas do *feeder*, não basta contar puramente com validações lógicas na camada da aplicação do servidor. É imperativa uma lógica sistêmica de banco de dados para prevenção à duplicação de anomalias temporais.

O mecanismo de *Upsert* constitui o ato conjugado e atômico de inserir um registro se este for perfeitamente novo na persistência e, em oposição, efetuar estritamente uma atualização no registro já contido caso este fira metadados imutáveis pré-condicionados (as chaves concorrentes primárias). No dialeto moderno do banco de dados PostgreSQL, invoca-se essa característica via instrução `INSERT ON CONFLICT DO UPDATE`. Neste projeto, isso foi feito utilizando a função `merge` do *Object-Relational Mapping* (ORM) (Object-Relational Mapping) do SQLAlchemy. Essa mescla transacional provê atualização sem carecer empenhar dispendiosos travamentos de linha (*row locks*) longos no processo.

Para que as consultas gRPC entreguem aos painéis dos pesquisadores gráficos perfeitamente sincronizados, isentos de contaminações das séries e saltos matemáticos fictícios, os barramentos de injeção aplicam em profundidade este corolário. Na eventualidade de os *feeders* retransmitirem parcelas horárias previamente já introduzidas, a abordagem *Upsert* consolida a consistência atômica. É, portanto, garantida a premissa de *Idempotên-*

*cia*: requisições idênticas com os mesmos parâmetros reproduzirão exatamente a mesma resultante estável a despeito das eventuais repetições impostas pelo emissor.

## 2.10 Autenticação e Autorização com JWT e Supabase

Garantir o livre mas vigiado intercâmbio de dados de mercado requer o uso de robustos arcabouços de gestão de identidade e acesso entre sistemas. O token JWT (*JSON Web Token*) fornece a assinatura unificada baseada em um corpo de formato JSON codificado que detém asserções (*claims*) prévias. Tais *claims* validam a legitimidade sem impor onerosas revisões sistêmicas (pesquisas na tabela de sessão do banco) em cada requisição transacionada via rede, seguindo fidedignamente o padrão aberto da indústria formulado no *Request for Comments* (RFC) 7519 do *Internet Engineering Task Force* (IETF) (**??**). Em outras palavras, realizamos autenticação utilizando processamento lógico, em vez de consulta a um banco de dados, o que é mais eficiente e escalável. Notavelmente, a obtenção de um JWT válido, um processo que é feito apenas uma vez por sessão, envolve um banco de dados. Isto não é problemático, pois uma sessão pode durar horas ou dias, conforme o critério do engenheiro que desenhou as configurações do JWT. Haja vista que o domínio do presente trabalho se concentra em redes, sistemas distribuídos, e dados financeiros, utilizamos uma solução de nível mais alto e maior simplicidade para lidar com o manejo e autenticação dos usuários. Mais especificamente, utilizamos o Supabase para gerenciar quem tem permissão para receber um token JWT válido.

Dessa forma, esses *tokens* são distribuídos por um *Backend as a Service* (BaaS) (*Backend as a Service*) (**??**). A implementação futura de Políticas de Segurança Orientadas a Linhas (*Row Level Security* - RLS) do PostgreSQL aliado ao gerenciamento contínuo das credenciais — sem sobrecarregar nosso código próprio de engenharia com tratamento complexo de senhas — representa o estado da arte das soluções para gerência de perfis e controle de acesso seguro.

Com essa proteção validada e autossuficiente via JWT, as interações entre os variados pesquisadores e o grupo diretor principal do LTI ganham controles finamente ajustados de permissão e isolamento granular. Ao invés da tradicional adoção de um gargalo de API Gateway obsoleto centralizando a autorização primária de forma bloqueante, os próprios microsserviços atestam localmente de forma criptográfica (na verificação de chaves assimétricas públicas) a permissão adequada requerida para leitura do *streaming* das cotações.

## 2.11 Desenvolvimento Incremental e Integração Contínua

Em contraponto às abordagens metodológicas clássicas e estritas de Cascata, metodologias contemporâneas ágeis pregam os aprimoramentos graduais focados em validação

rápida ao invés do dispêndio excessivo na abstração estática e antecipada do projeto. A essência do Desenvolvimento Incremental consiste no adorno iterativo de pequenos recursos (ou subcomponentes funcionais da aplicação) dispostos contínua e ordenadamente sobre uma fundação com forte padronização contratual nas interfaces de comunicação do sistema. Admito que uma preocupação quanto à Engenharia de Software é a possibilidade de que o desenvolvimento incremental se reduza ao mero trabalho *ad-hoc*.

No entanto, os avanços em ferramentas de linguagem natural têm avançado o desenvolvimento incremental a partir da criação de planos detalhados com mais facilidade, tanto antecipando problemas arquiteturais quando apontando ou criando possibilidades para evolução. Essa maleabilidade análise técnica do código e codificação aumentada (*augmented coding*), evoluíram a prática de *Continuous Integration* / Integração Contínua (CI). No mais, conforme a disponibilidade de tempo, uma lista de tarefas simples (*task list*) pode ser utilizada para determinar (especificar) a utilização de algumas técnicas de engenharia, tal como requerer a construção de um Diagrama de Classes bem feito para representar a estrutura do código, um Diagrama de Atividades para especificar o processo de autenticação e consumo de dados, e um Diagrama de Sequência, que é mais crítico, para representar a interação entre os micro-serviços do ecossistema. Tem se tornado mais claro que este último diagrama seria crítico, pois no início do projeto tínhamos apenas o qDATA, o *feeder* e o *dashboard*, porém durante a criação deste TCC emergiu no LTI a necessidade de um micro-serviço de simulação de correlações *bootstrapped*.

Esse novo cenário abre margem para a consolidação do uso de estratégias de orquestração de múltiplos repositórios (como submódulos e ferramentas de governança de código), um diferencial crucial no que tange à garantia de intercompatibilidade versional e à construção sustentável de ecossistemas de microsserviços. No Git, isso é feito com a função de submódulos, usando o comando `git submodule add` para adicionar um repositório dentro de outro. O submódulo é uma referência a um *commit* específico do repositório externo, e o Git mantém o controle dessa referência. Assim, quando alguém clona o repositório principal, ele pode usar o comando `git submodule update -init -recursive` para baixar os submódulos e garantir que as versões corretas sejam usadas.

Dessa forma, a adoção de um *pipeline* de CI robusto e automatizado, utilizando ferramentas como GitHub Actions, Jenkins ou GitLab CI/CD, é imperativa para o sucesso do projeto. O *pipeline* deve incluir etapas de construção, testes automatizados (unitários, de integração e de regressão), análise estática de código e validação de segurança. Isso assegura que cada alteração no código seja rigorosamente verificada antes de ser integrada ao ramo principal, mantendo a estabilidade e a qualidade do sistema à medida que ele evolui. Como dito antes, tal fato não era tão evidente no estado inicial do ecossistema, pois existem apenas três aplicações: (1) qDATA, (2) MT5-Websocket-Bridge, e (3) o *dashboard*. Todavia, à medida que o projeto cresce e mais componentes são adicionados – como o (4) Serviço de Simulações com Bootstrapping – a importância de um *pipeline* de CI bem

estruturado se torna ainda mais crítica para garantir a coesão e a integridade do sistema como um todo.

# 3 METODOLOGIA

A metodologia adotada neste trabalho foi estruturada para orientar o desenvolvimento de uma plataforma centralizada de recepção e distribuição de dados financeiros para o Laboratório de Tecnologias Inovadoras (LTI), com ênfase em baixa latência, disponibilidade. O estudo combina investigação bibliográfica, definição de requisitos a partir de um problema real do laboratório, desenvolvimento incremental da solução e proposta de validação por meio de testes funcionais e experimentos quantitativos.

## 3.1 Visão geral da metodologia

O ponto de partida desta pesquisa é a necessidade de responder à questão proposta anteriormente: como fornecer, de forma centralizada, sem dependências do MT5 e com baixa latência, dados em tempo real e históricos do ativo Mini-Índice Ibovespa para os pesquisadores do LTI ? A partir desse problema, a metodologia foi organizada como um processo prático e iterativo, no qual a arquitetura da solução e dos experimentos, as implementações técnicas e os testes de validação evoluem em conjunto.

Em vez de assumir um planejamento rígido e integral desde o início, como por exemplo um planejamento do tipo cascata, a proposta admite a incorporação gradual de requisitos e ajustes técnicos decorrentes das demandas advindas dos consumidores e das necessidades técnicas supervenientes. Tal dinamicidade exige, notavelmente, que haja um grau adequado de padronização de interfaces *back-end* e camadas-*controller* de dados no que diz respeito à API da plataforma.

Para além de questões de integração, esse *framework* é adequado porque muitos requisitos funcionais advêm de demandas de usuários reais do laboratório. Tais demandas, frequentemente, não são de conhecimento dos engenheiros. Em outras palavras, nós não temos pleno conhecimento do domínio-técnico ou domínio-funcional. Neste estudo, tal domínio consiste principalmente de estatística avançada, redes neurais e processos estocásticos. Assim, a metodologia visa um gerenciamento de projeto flexível o suficiente para acomodar diferentes necessidades, mas também suficientemente determinística para garantir a operabilidade contínua do ecossistema de serviços (*feeder*, qDATA, *dashboard*, etc).

## 3.2 Questões de pesquisa

Com base na pergunta principal do trabalho, foram definidas as seguintes questões de pesquisa, mais aderentes ao contexto do LTI:

1. Como estruturar uma infraestrutura única e interna de dados financeiros a qual centralize e abstraia as integrações de aquisição de dados (MT5, Polygon, etc)?

2. Quais componentes arquiteturais, protocolos de rede e *frameworks* de alto desempenho são necessários para disponibilizar dados históricos e em tempo real com baixa latência e alta disponibilidade? (Exemplos: gRPC, gRPC-Web, TimescaleDB, *websockets*, etc)

3. Como garantir a consistência e a atualização contínua dos dados ingeridos pela interface REST? (Exemplo: monitoração na interface REST com *webhooks*, canais de comunicação ininterruptos)

4. Quais testes e métricas quantitativas permitem verificar se a plataforma atende aos requisitos de desempenho e disponibilidades? (Exemplos: vazão, disponibilidade, latência média por megabyte)

Vale ressaltar que a plataforma não elimina completamente a dependência de fontes externas, mas sim abstrai o processo de obtenção dos dados, deixando as reais dependências a cargo exclusivo da própria API (qDATA) e do módulo *feeder* (como o mt5bridge). O objetivo é facilitar o trabalho dos pesquisadores ao remover o *overhead* de implementação de tecnologias como o MetaTrader 5 (MT5). Essa abstração ocorre não apenas por conveniência de desenvolvimento, mas para garantir que as aplicações analíticas rodem em modo *headless* em servidores baseados em Linux. Isso viabiliza estudos que dependem de operação contínua: mesmo quando o pesquisador encerra sua sessão ou fecha o computador pessoal, a aplicação objeto de sua pesquisa continua executando de maneira autônoma e recebendo dados. Sendo o qDATA projetado para uso estritamente interno do LTI, o seu consumo é feito exclusivamente por pesquisadores que necessariamente devem requerer credenciais junto à administração do projeto.

Essas questões guiarão tanto o projeto quanto a implementação da avaliação da solução. A resposta a cada uma delas depende da combinação entre escolha tecnológica, desenho da arquitetura e validação experimental.

## 3.3 Fontes de conhecimento e delimitação do estudo

Não fazem parte do escopo a execução automática de ordens, a gestão de risco de estratégias de *trading* e o desenvolvimento de ferramentas analíticas finais dos usuários. Nesse contexto, estamos criando uma solução de Market Data, isto é, provimento de dados de mercado em altíssima vazão em se tratando de baixar qualquer dado que já esteja no banco de dados – ainda que em quantidade grande (exemplo: 1Gb de dados) –. Por outro lado, está fora do escopo deste projeto prover dados extremamente atualizados (exemplo: dados que foram disponibilizados no *broker* original há apenas 5 microssegundos atrás)

com velocidade suficiente para execução de ordens de *trading* (Order Routing). Convém reiterar: proveremos altíssima vazão, e dados suficientemente recentes para Market Data, mas não suficientemente recentes para Order Routing. Felizmente, a construção de um eventual microsserviço de Order Routing para esse ecossistema é uma tarefa extremamente elegante de um ponto de vista de engenharia de software, e extremamente veloz de um ponto de vista técnico, porém não iremos abordar isso neste trabalho.

## 3.4 Desenvolvimento incremental da solução

A implementação foi conduzida de forma incremental, partindo de um esboço macro da arquitetura dos serviços responsáveis pela ingestão, armazenamento e distribuição dos dados. Essa visão macro consiste na definição de três principais pontos:

1. persistência de dados com PostgreSQL otimizado para séries temporais, utilizando as hyper-tabelas da extensão Timescale para armazenar os dados sem que a *performance* seja comprometida à medida que o banco de dados cresce (algo que ocorreria eventualmente em tabelas tradicionais de qualquer SGBD relacional);

2. A utilização de gRPC e gRPC-Web na camada de aplicação da rede para a construção de *endpoints* de alto desempenho voltados à distribuição de dados.

3. A utilização de um *end-point* REST/HTTPS na camada de aplicação da rede para ingestão de dados (alimentação do sistema de banco de dados)

### 3.4.1 Especificação do Contrato de Dados

Por fim, os requisitos de domínios impostos aos engenheiros nos levaram a armazenar a categoria de dados *Open, High, Low, Close, Volume* (OHLCV) (*Open, High, Low, Close, Volume*) para o ativo Mini-Índice Ibovespa, com granularidade de 1 minuto, e a frequência de atualização é de 1 minuto para os dados em tempo real. A escolha por essa categoria de dados se deve ao fato de que ela é amplamente utilizada em análises financeiras e estratégias de *trading*. A estrutura de dados (.proto) foi definida conforme o Código-fonte 3.1 apresentado a seguir:

Listing 3.1 – Estrutura da mensagem OHLCBar em Protocol Buffers

```
1 message OHLCBar {
2     // Optional fields
3     optional int64 id = 1;
4     optional string symbol = 2;
5     optional string timeframe = 3;
6     optional string source_id = 4;
```

```
7       optional int64 ingested_at = 5; // Unix timestamp
8
9       // Required fields
10      int64 time = 6; // Unix timestamp in seconds
11      string time_utc = 7; // \gls{ISO} 8601 \gls{UTC}
12      string time_sp = 8; // \gls{ISO} 8601 Sao Paulo
13      double open = 9;
14      double high = 10;
15      double low = 11;
16      double close = 12;
17      int64 tick_volume = 13;
18      int64 spread = 14;
19      int64 real_volume = 15;
20  }
```

A solução é implementada em módulos, de modo que cada parte possa ser testada isoladamente antes da integração completa. A arquitetura é orientada à separação de responsabilidades, para que a evolução de um componente não comprometa os demais, e nesse sentido faz uso amplo de *containers* Docker. Desta forma: conseguimos construir três microsserviços: o *feeder*, responsável pela ingestão de dados; o qDATA, responsável pela persistência e consulta dos dados; e o *dashboard*, que não é o foco deste trabalho, mas apenas porque não somos os donos desse produto ('*product ownership*'), e sua existência é para nós uma fonte de demandas ou fonte de requisitos.

Uma regra de *design* importante a ser integrada à arquitetura do presente ecossistema – o ecossistema em que o qData está inserido – é o princípio da independência de dados, descrito por Chris Richardson (**??**). Essa abordagem sugere que sistemas de banco de dados concentrados em um único nó com a finalidade de servir módulos cujos propósitos são distintos gerariam problemas de acoplamento complicados e, sobretudo, desnecessários. Ele discorre:

> Uma característica chave da arquitetura de microsserviços é que os serviços são fracamente acoplados e se comunicam apenas via APIs [...] Em tempo de desenvolvimento, os desenvolvedores podem alterar o esquema de um serviço sem precisarem coordenar com desenvolvedores que trabalham em outros serviços. Em tempo de execução, os serviços são isolados uns dos outros — por exemplo, um serviço nunca será bloqueado porque outro serviço detém um bloqueio no banco de dados.

Em termos práticos, isso implica, por exemplo, a utilização de um banco de dados no serviço qData (PostgreSQL) e um outro banco de dados próprio no serviço de cálculos (SQLite). Embora pudéssemos pensar em armazenar no qData dados numéricos provenientes de cálculos estatísticos – baseado na ideia de um "centro de dados" –, o princípio em discussão recomenda um banco de dados próprio para cada serviço, preferencialmente

rodando em contêineres. Nesse sentido, podemos argumentar que um nó é uma API associada a um sistema de banco de dados independente.

## 3.5 Governança

A governança da arquitetura de microsserviços é fundamental para garantir a manutenibilidade, a segurança e a evolução padronizada do sistema. Esta seção descreve as principais diretrizes e políticas adotadas no contexto deste trabalho.

### 3.5.1 Versionamento de API

Para garantir a retrocompatibilidade e permitir a evolução contínua dos serviços sem impactar os clientes existentes, adota-se o versionamento de APIs. O versionamento é realizado explicitamente através do caminho na URL da API (por exemplo, `/api/v1/recurso` e `/api/v2/recurso`). Quando ocorrem mudanças que quebram a compatibilidade atual (*breaking changes*), uma nova versão principal (*major*) é disponibilizada, mantendo-se a versão anterior ativa por um período de transição pré-estabelecido (período de *sunset*).

### 3.5.2 Políticas de Acesso por Perfil

O controle de acesso é gerenciado com base em perfis e papéis (*Role-Based Access Control* - RBAC). A autenticação e a autorização são tratadas de forma centralizada (geralmente por meio de um API Gateway ou serviço de identidade dedicado), e as requisições repassadas aos microsserviços carregam as informações de acesso através de *tokens* seguros, como JWT (*JSON Web Tokens*). Os perfis determinam o nível de permissão (como leitura, escrita ou administração) de cada usuário, garantindo que as políticas de menor privilégio sejam aplicadas nas operações.

### 3.5.3 Ownership dos Microsserviços

Para garantir a responsabilidade e facilitar a escalabilidade organizacional, adota-se o conceito de propriedade (*ownership*) bem definida. Cada microsserviço possui uma equipe responsável por todo o seu ciclo de vida, desde a concepção até a operação em produção (modelo *build it, run it*). Essa abordagem reduz os gargalos de comunicação, garante a *accountability* e assegura que a equipe tenha total domínio técnico e de negócio sobre o serviço pelo qual é responsável.

### 3.5.4 Gestão de Mudança

O processo de gestão de mudança estabelece que novas implementações, refatorações e correções sejam integradas de maneira segura e controlada. O fluxo de desenvolvimento

apoia-se em práticas de Integração e Entrega Contínuas (CI/CD). Toda modificação no código passa por validações automáticas que incluem testes de unidade, testes de integração e análise estática de código. Além disso, mudanças arquiteturais e atualizações em contratos de API exigem revisão por pares (*code review*) obrigatória antes da integração na ramificação principal (*main branch*) e posterior promoção aos ambientes de homologação e produção.

## 3.6 Técnicas e testes aplicados

Para responder às questões de pesquisa, a metodologia combina técnicas de engenharia de software e validação experimental. Os testes funcionais verificam se os componentes executam corretamente as tarefas esperadas, como autenticação, ingestão, armazenamento e consulta. Os testes de integração avaliam a comunicação entre serviços e o comportamento da plataforma como um todo. Já os testes de desempenho medem latência, tempo de resposta, vazão e estabilidade sob carga, permitindo verificar se a centralização dos dados efetivamente reduz o esforço individual dos pesquisadores sem comprometer a experiência de uso.

Além disso, o *blueprint* deste projeto propõe testes automatizados na etapa de integração contínua (combinação de *workflow* do GitHub com *scripts shellscript*) visando manter a consistência do sistema durante sua evolução. **Importante:** esses testes são apenas propostas e **não foram implementados** no presente trabalho. Eles têm o objetivo específico de garantir a robustez da construção e do desenvolvimento, mas sua execução permanece como futuro trabalho.

## 3.7 Métricas de avaliação

No eixo de desempenho, mediremos latência média, variabilidade da latência, tempo de resposta, vazão e capacidade de atendimento simultâneo. No eixo de confiabilidade, analisaremos a disponibilidade dos serviços, a estabilidade da ingestão, além de que realizaremos testes de carga com o *framework* Locust.

## 3.8 Procedimento experimental

O procedimento experimental é composto pelas seguintes etapas:

1. levantamento da necessidade do laboratório e definição das questões de pesquisa;
2. implementação incremental dos serviços de ingestão, persistência, autenticação e distribuição;

3. elaboração e execução de testes funcionais por meio de *scripts* de Python com GitHub Actions e *shellscript*

4. elaboração de testes de integração também via *scripts* de Python com GitHub Actions e *shellscript*

5. coleta e análise dos resultados obtidos, com comparação entre o comportamento esperado e o comportamento observado, e posterior exibição e discussão dos resultados

Esse fluxo permite associar cada decisão de projeto a evidências observáveis, tornando a metodologia coerente com o caráter aplicado do trabalho.

## 3.9 Ameaças à validade

A avaliação de sistemas computacionais aplicados está sujeita a limitações metodológicas que podem influenciar a interpretação dos resultados. Nesta seção, discutem-se as principais ameaças identificadas, classificadas em ameaças à validade interna e à validade externa (generalização).

### 3.9.1 Validade Interna

Entre as principais ameaças aos objetivos de nível de qualidade de serviço (*Service Level Objective* – SLO) e às medições de desempenho, destacam-se a dependência de serviços externos para obtenção dos dados (como o MetaTrader 5 e conexões de corretoras), a variabilidade estocástica das redes de telecomunicação e as disparidades entre o ambiente de desenvolvimento e o ambiente de produção.

Para mitigar tais fatores, os experimentos quantitativos foram desenhados com critérios rigorosos de reprodutibilidade: emprego de técnicas de aquecimento (*warmup*) de conexões para neutralizar o impacto de *handshakes* TCP/TLS e resolução de DNS, intercalação sistemática de requisições entre os protocolos avaliados e execução em instâncias dedicadas de teste com cargas controladas.

### 3.9.2 Validade Externa e Generalização

A principal ameaça à validade externa reside no escopo amostral e no contexto de validação do sistema:

1. **Amostra restrita e perfil dos usuários:** A plataforma foi validada no ecossistema específico do Laboratório de Tecnologias Inovadoras (LTI), contando com a adoção prática de um número reduzido de pesquisadores e estudantes de engenharia e ciência da computação. Embora esse público represente com fidelidade os usuários-alvo imediatos do projeto (pesquisadores de finanças quantitativas), suas habilidades

técnicas e padrões de consulta via *scripts* analíticos (como em Python) podem não refletir integralmente as demandas de usuários de outros perfis ou ambientes corporativos de larga escala.

2. **Regime de carga e concorrência:** No contexto operacional do LTI, as requisições ocorrem predominantemente em lotes assíncronos ou execuções programadas de experimentos. Dessa forma, a validação com usuários reais não submeteu o sistema a uma concorrência massiva de múltiplos clientes acessando simultaneamente transmissões em tempo real sob alta contenção de recursos. Para mitigar essa limitação empírica com usuários humanos, a metodologia incorporou ensaios experimentais sintéticos com diferentes volumetrias de carga (cenários de baixo, médio e alto volume de barras).

3. **Representatividade dos ativos financeiros:** A avaliação prática concentrou-se no contrato futuro de Mini-Índice Ibovespa, que possui alta liquidez e volume concentrado. Embora a modelagem do banco de dados relacional temporal (TimescaleDB) suporte genericamente múltiplos ativos e granularidades temporais, a operação com centenas de ativos concorrentes simultâneos exigirá testes adicionais de capacidade de ingestão e particionamento em trabalhos futuros.

Para favorecer a transferibilidade da solução para outros grupos de pesquisa ou instituições congêneres, a plataforma adotou padrões abertos de comunicação (gRPC e HTTPS RESTful), conteinerização completa dos serviços via Docker e autenticação desacoplada baseada em JSON Web Tokens (JWT). Uma eventual replicação do ecossistema requer apenas o provisionamento de instâncias autônomas com configuração independente de credenciais e provedores de mercado, resguardando a integridade e compatibilidade através do versionamento explícito de rotas da API.

## 3.10 Síntese

Em síntese, a metodologia combina fundamentação teórica, desenvolvimento incremental e validação experimental para construir uma solução centralizada de dados financeiros voltada ao LTI. O foco do trabalho está em demonstrar que é possível oferecer acesso compartilhado a dados históricos e em tempo real com baixa latência, mantendo consistência, disponibilidade e reduzindo o esforço individual de integração por parte dos pesquisadores. Por fim, a metodologia visa aplicar os conhecimentos da grade curricular do Bacharelado em Engenharia de Software oferecido pelo Campus de Russas da Universidade Federal do Ceará, uma oportunidade de aplicação de técnicas de engenharia topo de linha. Definitivamente, esta metodologia é uma tentativa de orientar a construção de uma solução que atenda às exigências elementares de um projeto de engenharia de software profissional.

# 4 CRONOGRAMA

O desenvolvimento deste Trabalho de Conclusão de Curso foi estruturado e executado ao longo do primeiro semestre de 2026. As atividades iniciaram-se de maneira concomitante ao início do semestre letivo e da disciplina de Projeto de Pesquisa Científica e Tecnológica. A Tabela 1 consolida os principais marcos temporais e as fases de evolução do projeto arquitetural do qDATA, desde o delineamento inicial até a sua entrega e finalização no dia atual.

Tabela 1 – Cronograma de execução das atividades do TCC

| **Período** | **Atividade** |
|---|---|
| 05 de Janeiro | Início do semestre letivo, da disciplina de Projeto de Pesquisa Científica e Tecnológica e definição do escopo macro do LTI FinHub. |
| Janeiro | Revisão bibliográfica focada em arquitetura de microsserviços, bancos de dados de séries temporais e protocolos de comunicação. |
| Fevereiro | Preparação da infraestrutura com Docker, conteinerização de dependências. |
| Março | Desenvolvimento da API assíncrona (REST) utilizando FastAPI e integração com o banco de dados TimescaleDB. |
| Abril | Implementação da arquitetura gRPC para streaming de dados bidirecional, e configuração de autenticação via JWT/Supabase. |
| Maio | Testes de carga, validação da idempotência (*Upsert*), e início da escrita dos capítulos textuais (Introdução e Fundamentação). |
| Junho | Finalização da redação da Metodologia, revisão dos textos produzidos e enquadramento nas normas da ABNT. |
| 22 de Junho | Revisão final, encerramento do desenvolvimento e entrega formal do documento do TCC. |

# 5 RESULTADOS

A condução deste trabalho proporcionou resultados tanto técnicos quanto organizacionais e de adoção, que são detalhados a seguir.

## 5.1 Análise quantitativa de *performance* de comunicação

Para avaliar empiricamente as vantagens de cada protocolo implementado, foram conduzidos testes comparando a abordagem HTTPS RESTful tradicional, gRPC-Web e gRPC puro. Os ensaios levaram em conta fatores de distorção, tais como como o aquecimento de conexões (DNS, *Handshake* TCP e TLS/SSL) e a flutuação natural de latência de rede, empregando técnicas de *warmup* e intercalação de requisições. Para fins práticos de análise, os testes foram focados em contrastar duas situações diametralmente opostas de demanda: o baixo volume cotidiano e o tráfego de maior volume de dados.

### 5.1.1 Cenário 1 — Baixo Volume

O primeiro cenário simula respostas de carga pequena representando consultas de poucos dias. Este é o caso de uso mais comum na utilização diária da API por clientes finais. A latência média observada revelou tempos de resposta muito baixos para todos os protocolos.

Tabela 2 – Cenário 1 — Baixo Volume: latência média agregada (poucos dias).

| **Métrica** | **HTTPS** | **gRPC-Web** | **gRPC Puro** |
|---|---|---|---|
| Latência média (ms) | 45,2 | 38,5 | 30,1 |

### 5.1.2 Cenário 2 — Volume Intermediário (Aleatório A)

O segundo cenário simula consultas de volume intermediário, representando janelas de pregão mais extensas. Cada execução retornou **301 barras** OHLC M1 — correspondendo a aproximadamente 5 horas de dados de mercado. O aumento de volume permite observar como cada protocolo escala a partir da latência de base observada no Cenário 1, especialmente no que diz respeito ao throughput e ao tamanho do payload.

Tabela 3 – Cenário 2 — Volume Intermediário: métricas agregadas (301 barras/execução).

| **Métrica** | **HTTPS** | **gRPC-Web** | **gRPC Nativo** |
|---|---|---|---|
| Latência média (ms) | 207,19 | 314,18 | 241,69 |
| Latência mediana (ms) | 187,37 | 342,50 | 238,01 |
| Percentil 95 (ms) | 296,83 | 394,59 | 287,05 |
| Percentil 99 (ms) | 296,83 | 394,59 | 287,05 |
| Latência mínima (ms) | 174,21 | 209,40 | 210,18 |
| Latência máxima (ms) | 296,83 | 394,59 | 287,05 |
| Throughput médio (bars/s) | 1508,08 | 1012,86 | 1258,20 |
| Total de barras recebidas | 1.505 | 1.505 | 1.505 |
| Barras por execução | 301 | 301 | 301 |
| Taxa de erro | 0 | 0 | 0 |

### 5.1.3 Cenário 3a — Alto Volume (Janela de 2 meses)

Este cenário introduz um alto volume de dados com uma janela de consulta de aproximadamente 2 meses (01/01/2026 a 01/03/2026). Cada execução retornou **16.865 barras** OHLC M1. Neste contexto, a diferença arquitetural entre os protocolos se torna mais evidente: o HTTPS/REST necessita de múltiplas requisições paginadas para obter o conjunto completo de dados, enquanto os protocolos gRPC utilizam *server-streaming* em uma única chamada.

A análise empírica deste cenário demonstra de forma contundente a superioridade do gRPC no tráfego para requisições maiores. Conforme observado na Tabela 4, a latência média do gRPC nativo ( 4.264 ms) foi equivalente a quase metade da latência observada no protocolo HTTPS ( 8.145 ms), refletindo ganhos expressivos de throughput e eficiência.

Tabela 4 – Cenário 3a — Alto Volume (2 meses): métricas agregadas (16.865 barras/execução).

| **Métrica** | **HTTPS** | **gRPC-Web** | **gRPC Nativo** |
|---|---|---|---|
| Latência média (ms) | 8145,18 | 4259,45 | 4264,11 |
| Latência mediana (ms) | 7219,69 | 3993,21 | 4150,32 |
| Percentil 95 (ms) | 11 877,38 | 5522,02 | 4837,24 |
| Percentil 99 (ms) | 11 877,38 | 5522,02 | 4837,24 |
| Latência mínima (ms) | 6963,56 | 3473,75 | 4035,85 |
| Latência máxima (ms) | 11 877,38 | 5522,02 | 4837,24 |
| Throughput médio (bars/s) | 2156,82 | 4070,72 | 3972,01 |
| Total de barras recebidas | 84.325 | 84.325 | 84.325 |
| Barras por execução | 16.865 | 16.865 | 16.865 |
| Taxa de erro | 0 | 0 | 0 |

## 5.2 Avaliação Qualitativa dos Protocolos de Comunicação

Durante o desenvolvimento, observou-se que alguns aspectos da arquitetura e modelagem propostas se comportaram de forma diferente da esperada sob condições reais. Nesse sentido, a tentativa de utilização do protocolo gRPC para propiciar uma transferência de dados mais veloz mostrou-se inadequada para a API pública, uma vez que as séries temporais diárias solicitadas não ultrapassaram alguns kilobytes de tamanho.

A esse respeito, o protocolo HTTPS tradicional demonstrou superioridade por diversos motivos: (1) possibilitou uma implementação muito mais fácil nos ambientes locais dos clientes; (2) a curva de aprendizado para discentes iniciantes que desejavam consumir a API apresentou-se consideravelmente menor; e (3) o tempo gasto de preparação ("*overhead*"de comunicação) para que o cliente começasse a receber os dados do servidor foi significativamente melhor no HTTPS em comparação ao gRPC-Web.

### 5.2.1 Adoção pelos Usuários e Impacto Prático

O projeto cumpriu sua proposta de prover acesso de alta disponibilidade a dados para os membros internos do laboratório, provando-se útil na prática. O número de clientes consumindo os dados do sistema por meio de experimentos — frequentemente implementados através de *scripts* focados em análises matemáticas e estatísticas, e não estritamente em engenharia de software — aumentou gradualmente.

Além disso, o serviço passou a ser adotado sistematicamente por aplicações em nuvem hospedadas no laboratório. Como exemplos práticos de adoção bem-sucedida, destacam-se a aplicação de cálculo de correlações com *bootstrap*, sob a condução de Marcos Muniz, e os experimentos estatísticos realizados pelo discentes Vitor. Como resultado dessa experiência, atualmente, emergiu um estudo por parte de discentes do projeto com o intuito de produzir uma API de alto desempenho baseada no qData, com granularidade de dados na ordem de microssegundos.

### 5.2.2 Impactos Organizacionais e na Infraestrutura

No aspecto abstrato e organizacional, o presente projeto de software serviu como um importante ponto de partida para o cultivo de uma cultura de engenharia de software de alto padrão no ambiente do LTI. Desse desejo de rigor formal originaram-se diversas iniciativas paralelas:

- A implantação de um roteador (*proxy* reverso) de alto nível no servidor do projeto (Traefik), centralizando e garantindo segurança e roteamento adequado das requisições externas;
- A configuração de um sistema de código aberto (*open-source*) (Bitwarden) para o manejo seguro de credenciais, senhas e segredos altamente sensíveis do projeto;

- O início da estruturação organizacional do LTI Finanças, com a introdução formal de cargos de administração e gerenciamento de projetos;
- A criação de documentações de integração (*onboarding*) e de descrição estruturada para novos membros do projeto.

### 5.2.3 Experiência adquirida

A experiência adquirida durante o projeto foi tanto individual quanto coletiva. No individual âmbito, exigiu disciplina de trabalho (ou "work ethic"), a lide com as demandas do LTI enquanto stake-holder cliente, e o aprendizado e aplicação contínuo de novas tecnologias e técnicas. A lide com técnicas novas foi tão frequente quanto a lide com técnicas que já conhecíamos. No âmbito coletivo, a comunicação e relacionamento com discentes colaboradores, que atuaram diretamente no repositório do qData foi considerável, e exigiu sensibilidade para estabelecer prazos e atribuir tarefas a outros discentes, bem como acompanhar o andamento dessa tarefa. Todavia, houve momentos em que, por inexperiência e falta de sensibilidade, eu atribui tarefas que tinham um nível de complexidade incompatível com o nível de experiência de alguns discentes, uma questão que se deu em um cenário de transição da Ciência da Computação e da Engenharia de Software. Nesse sentido, verificamos um momento histórico extremamente dinâmico e incerto.

# 6 CONCLUSÕES E TRABALHOS FUTUROS

Este capítulo encerra o trabalho, retomando os objetivos propostos e discutindo os resultados obtidos em diversas frentes (técnica e organizacional), além de sugerir direções para projetos subsequentes.

## 6.1 Considerações Finais e Cumprimento dos Objetivos

A partir do estudo acadêmico realizado e do trabalho dedicado a este sistema enquanto atividade laboral no LTI, conclui-se que o projeto de criação de um núcleo de informações centralizado provou-se extremamente produtivo e altamente útil para seus usuários, sanando uma dor real dos pesquisadores no acesso a séries financeiras.

## 6.2 Análise Técnica: Performance vs. Conveniência

Como visto, o uso do protocolo gRPC enquanto interface de API externa voltada a clientes finais (pesquisadores) não trouxe os benefícios práticos esperados em requisições cotidianas de baixo volume. O pequeno volume de dados e o *overhead* de desenvolvimento (construção do código inicial) não compensam, em termos absolutos, a baixíssima curva de aprendizado e a conveniência do protocolo HTTPS.

No entanto, em cenários de alta demanda e tráfego massivo, a latência média do gRPC reduziu-se à metade em comparação ao HTTPS, comprovando reais ganhos de *throughput*. Fica claro que existe um forte *trade-off* entre a conveniência para prototipação (HTTPS) e o alto desempenho de rede (gRPC).

## 6.3 Impactos Organizacionais no LTI Finanças

Em relação ao impacto estrutural, as atividades formais de engenharia de software introduzidas evidenciaram à liderança que novas estruturas de gerenciamento são estritamente necessárias para a maturidade do laboratório. Verificou-se que uma estrutura hierárquica simples (um tech lead, um gestor e demais membros técnicos) funcionou melhor que tentativas de organogramas excessivamente robustos e complexos.

Visualiza-se a forte necessidade de refinar os processos formais de *onboarding* e documentação. Atualmente, a organização encontra-se em um nível de maturidade Parcialmente Gerenciado (PM), sendo fundamental o estabelecimento de processos bem documentados

para alcançar um nível "Gerenciado"(G), garantindo uma maior Orientação a Resultados (OR).

## 6.4 Trabalhos Futuros

Como trabalhos futuros, propõem-se as seguintes iniciativas:

- **Uso Estratégico do gRPC:** Recomenda-se aplicar o gRPC exclusivamente na comunicação interna de altíssimo rendimento entre microsserviços (como entre *backends* e bancos de dados, sem exposição à Internet), evitando seu uso na prototipação de experimentos em Python ou para alimentar *dashboards* web.
- **Expansão de Dados e Recursos:** Ampliar os conjuntos de dados (novos ativos financeiros e granularidades intradiárias/micro-segundos) e aprimorar continuamente o controle granular de acesso (Supabase).
- **Evolução com Inteligência Artificial:** Aproveitar o ganho de produtividade gerado pelos Modelos de Linguagem Grande (LLMs) para acelerar a expansão da API qData, atendendo a novas iniciativas como o emergente "Núcleo de Investimentos e Especulação Financeira"(NIEF).